

5.观察空间

观察空间推导

在unity中观察空间使用右手坐标系

观察空间(view space)也叫摄像机空间(camera space)。在观察空间，摄像机位于原点，坐标轴理论上说可以任意选取，unity默认选择的是右手坐标系。x轴指向右方，y轴指向上方，z轴指向摄像机后方。

unity在模型空间和世界空间中Z指向的都是物体的前方。因为这两者都是左手坐标系。但是观察空间中使用的是右手坐标系。

观察空间中，摄像机的正前方指的-Z轴方向，符合opengl传统。

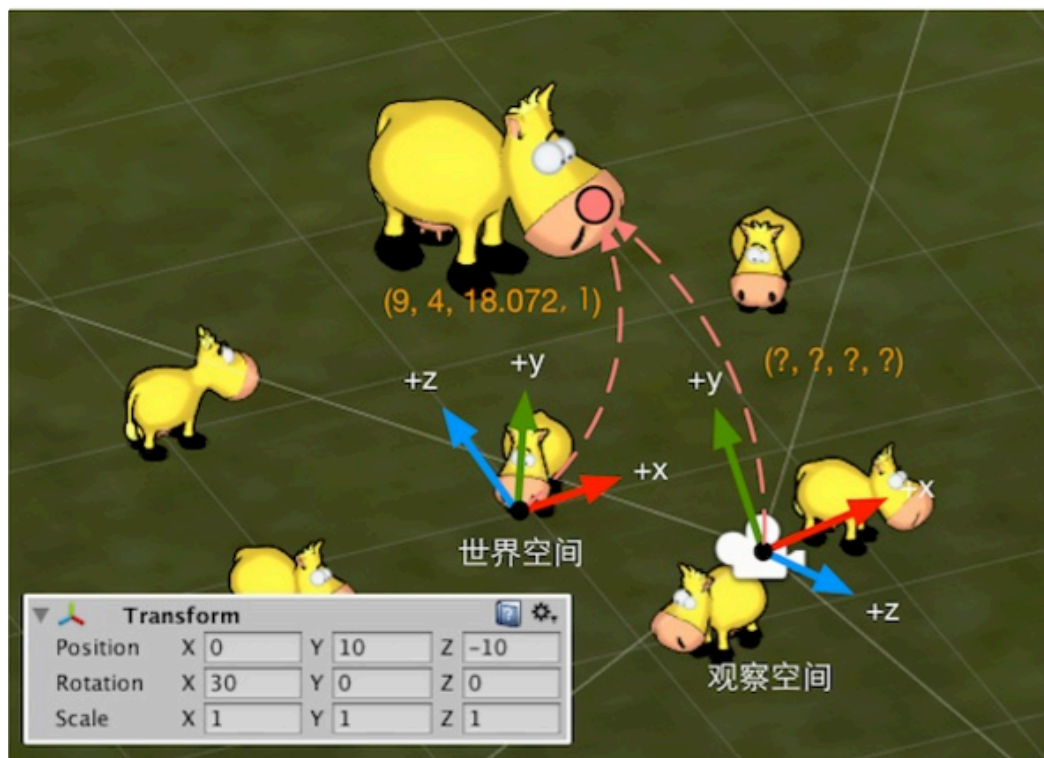
观察空间不是屏幕空间，观察空间是三维空间，屏幕空间是二维空间。观察空间到屏幕空间需要经过“投影 (projection) ”。

观察空间推导

其实观察变换(view transform)的推导,可以假设成，把摄像机进行逆向变换。

摄像机在世界空间中的变换是先按(30, 0, 0)进行旋转，然后按(0, 10, -10)进行了平移。那么，为了把摄像机重新移回到初始状态（这里指摄像机原点位于世界坐标的原点、坐标轴与世界空间中的坐标轴重合），我们需要进行逆向变换，即先按(0, -10, 10)平移，以便将摄像机移回到原点，再按(-30, 0, 0)进行旋转，以便让坐标轴重合。因此，变换矩阵就是：

$$\begin{aligned}
 M_{visw} &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.866 & 0.5 & 0 \\ 0 & -0.5 & 0.866 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -10 \\ 0 & 0 & 1 & 10 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.866 & 0.5 & -3.66 \\ 0 & -0.5 & 0.866 & 13.66 \\ 0 & 0 & 0 & 1 \end{bmatrix}
 \end{aligned}$$



但是，由于观察空间使用的是右手坐标系，因此需要对z分量进行取反操作。我们可以通过乘以另一个特殊的矩阵来得到最终的观察变换矩阵：

$$\begin{aligned}
 M_{w \rightarrow w} &= M_{nsgats} P_{visw} \\
 &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.866 & 0.5 & -3.66 \\ 0 & -0.5 & 0.866 & 13.66 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.866 & 0.5 & -3.66 \\ 0 & 0.5 & -0.866 & -13.66 \\ 0 & 0 & 0 & 1 \end{bmatrix}
 \end{aligned}$$

我们可以用它来对妞妞的鼻子进行顶点变换了：

$$\begin{aligned}
 P_{w \rightarrow w} &= M_{visw} P_{world} \\
 &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.866 & 0.5 & -3.66 \\ 0 & 0.5 & -0.866 & -13.66 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 9 \\ 4 \\ 18.072 \\ 1 \end{bmatrix} = \begin{bmatrix} 9 \\ 8.84 \\ -27.31 \\ 1 \end{bmatrix}
 \end{aligned}$$

在世界空间下，妞妞鼻子的位置是(9, 4, 18.072)

我们就得到了观察空间中妞妞鼻子的位置——(9, 8.84, -27.31)。

在opengl一些代码中，观察空间不是这么定义的

```

1  glm::mat4 view = glm::lookAt(glm::vec3(0,0,10), glm::vec3(0,0,0), glm::vec3(0,1,0));
2  if(!hasPrint) {
3      std::cout << "viewMat:" << std::endl;
4      printMatrix(view);
5  }
6  glm::lookAt函数的参数分别是相机的位置 (glm::vec3(0,0,10)) ,
7      目标的位置 (glm::vec3(0,0,0)) , 和上向量 (glm::vec3(0,1,0)) 。
8      这段代码定义了一个位于(0,0,10)的位置，并朝向原点(0,0,0)的相机。

```

```

1  #include <glm/glm.hpp>
2  #include <glm/gtc/matrix_transform.hpp>
3  #include <iostream>
4
5  void printMatrix(const glm::mat4& matrix) {
6      for (int i = 0; i < 4; ++i) {
7          for (int j = 0; j < 4; ++j) {
8              std::cout << matrix[i][j] << " ";
9          }
10         std::cout << std::endl;
11     }
12 }
13
14 int main() {
15     // Camera parameters
16     glm::vec3 cameraPosition(0.0f, 10.0f, -10.0f);
17     glm::vec3 target(0.0f, 0.0f, 0.0f);
18     glm::vec3 up(0.0f, 1.0f, 0.0f);
19
20     // Define the rotation angle around the x-axis
21     float angleX = glm::radians(30.0f); // convert degrees to radians
22
23     // Calculate the direction vector
24     glm::vec3 direction = target - cameraPosition;
25
26     // Create rotation matrix
27     glm::mat4 rotation = glm::rotate(glm::mat4(1.0f), angleX, glm::vec3(1.
0f, 0.0f, 0.0f));
28
29     // Apply rotation to the direction and up vector
30     glm::vec3 rotatedDirection = glm::vec3(rotation * glm::vec4(directio
n, 0.0f));
31     glm::vec3 rotatedUp = glm::vec3(rotation * glm::vec4(up, 0.0f));
32
33     // Calculate the new target position
34     glm::vec3 newTarget = cameraPosition + rotatedDirection;
35
36     // Create the view matrix using the rotated position and up vector
37     glm::mat4 view = glm::lookAt(cameraPosition, newTarget, rotatedUp);
38
39     std::cout << "viewMat:" << std::endl;
40     printMatrix(view);
41
42     return 0;
43 }
44

```

45 相机参数：定义相机的位置、目标和上向量。

46 旋转角度：将30度转换为弧度。

47 方向向量：计算相机的方向向量，即目标点减去相机位置。

48 旋转矩阵：创建绕 X 轴旋转的矩阵。

49 应用旋转：将旋转应用于方向向量和上向量，计算出旋转后的方向和上向量。

50 新目标位置：计算旋转后的目标位置。

51 视图矩阵：使用 `glm::lookAt` 函数，传入相机位置、新的目标位置和旋转后的上向量来创建视图矩阵。

52

